

Table of Contents

Introduction.....	2
npm tools.....	3
<i>npm install: install node modules in package.json.....</i>	<i>3</i>
<i>npm run build: build the project.....</i>	<i>3</i>
<i>npm start: build and run the frontend project.....</i>	<i>4</i>
Use the Browser built-in Developer Tools.....	5
Use the browser extension React Developer Tools.....	7

Demo: React Debugging

Author: Baifan Zhou

Detailed explanations are created with the help of ChatGPT.

Introduction

npm Tools

- **npm install:** Installs node modules listed in package.json. Reinstalls missing modules if needed.
- **npm run build:** Builds the project and generates the build folder. Displays warnings and errors (e.g., unused imports).
- **npm start:** Builds and runs the frontend project, displaying warnings and errors in the terminal and browser.

Browser Built-in Developer Tools

- **Elements:** Inspect components and styles on the web page.
- **Console:** View logs and outputs from the code.
- **Sources:** Set breakpoints and inspect variables. Use buttons and scopes to control debugging.

React Developer Tools Extension

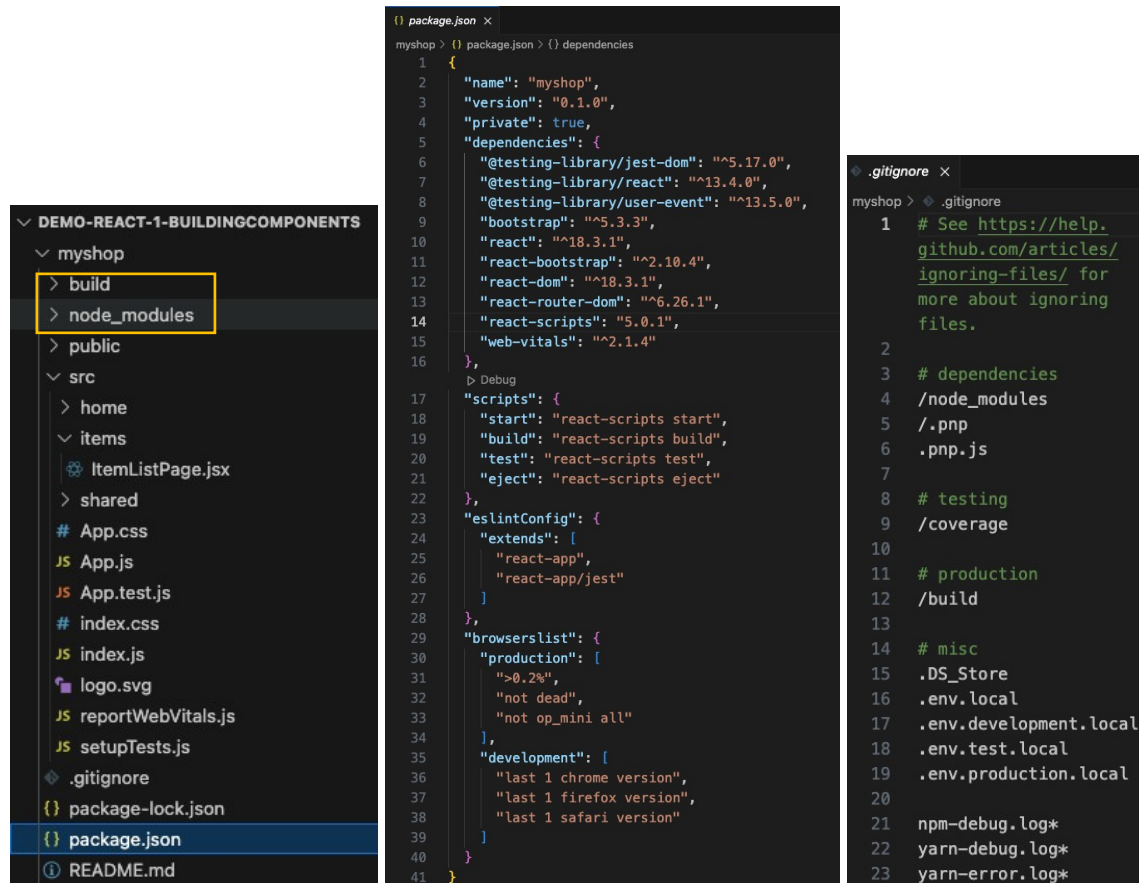
- **Components:** View component tree, inspect and modify props and states, and use the element picker.
- **Profiler:** Measure and analyse rendering performance of React components.

npm tools

The npm, node package manager, offers many tools.

npm install: install node modules in package.json

The node_modules can be store all the installed modules documented in package.json. If some modules are missing, or the complete folder is missing. Run `npm install` to reinstall it.

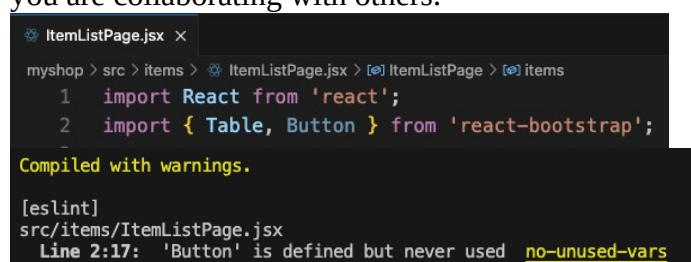


npm run build: build the project

The folder build stores intermediate build files. Run `npm run build` to build the project, (which generates the folder build), and see potential warnings or errors.

For example, I imported Button from react-bootstrap, but never used it, `npm run build` will show the warnings.

Both the folder node_modules and build do not need to be shared to others when you are collaborating in a team on different parts of the project. The node_modules can be installed. The build files can change each time the project is changed. Both can be put in .gitignore if you are collaborating with others.



npm start: build and run the frontend project

In most cases, you don't need to run `npm run build`, because `npm start` already builds and runs the frontend project. The warnings and errors are also shown in the terminal.

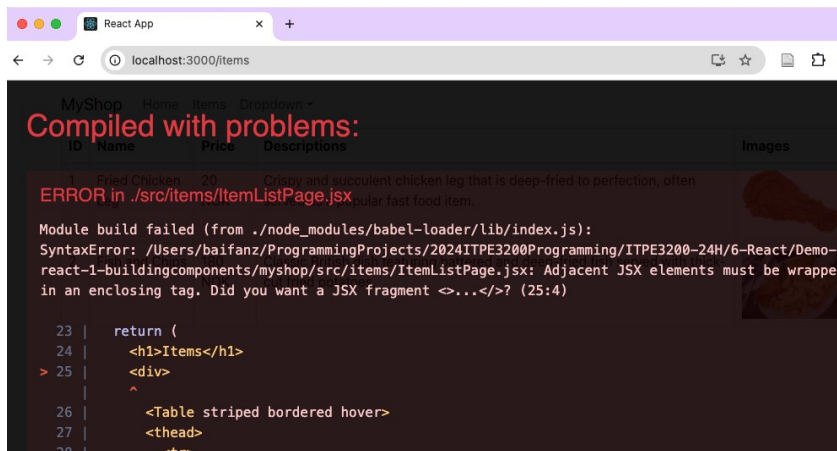
In case of an error (e.g., I put the `<h1>` out of the `<div>` in the return of `ItemListPage.jsx`. VS Code will show the error.

```
23 |   return (
24 |     <h1>Items</h1>
25 |     <div>
26 |       <Table striped bordered hover>
27 |         <thead>
28 |           <tr>
29 |             <th>ID</th>
30 |             <th>Name</th>
31 |             <th>Price</th>
32 |             <th>Descriptions</th>
33 |             <th>Images</th>
34 |           </tr>
35 |         </thead>
36 |         <tbody>
37 |           {items.map(item => (
38 |             <tr key={item.ItemId}>
39 |               <td>{item.ItemId}</td>
40 |               <td>{item.Name}</td>
41 |               <td>{item.Price} NOK</td>
42 |               <td>{item.Description}</td>
43 |               <td><img src={item.ImageUrl} alt={item.Name} width="120" /></td>
44 |             </tr>
45 |           ))}
46 |         </tbody>
47 |       </Table>
48 |     </div>
49 |   );
50 | };
```

The terminal will show the error:

```
SyntaxError: /Users/baifanz/ProgrammingProjects/2024ITPE3200Programming/ITPE3200-24H/6-React/Demo-react-1-buildingcomponents/myshop/src/items/ItemListPage.jsx: Adjacent JSX elements must be wrapped in an enclosing tag. Did you want a JSX fragment <>...</>? (25:4)
> ...</>? (25:4)
23 |   return (
24 |     <h1>Items</h1>
> 25 |     <div>
    |     ^
26 |       <Table striped bordered hover>
27 |         <thead>
28 |           <tr>
ERROR in ./src/items/ItemListPage.jsx
Module build failed (from ./node_modules/babel-loader/lib/index.js):
SyntaxError: /Users/baifanz/ProgrammingProjects/2024ITPE3200Programming/ITPE3200-24H/6-React/Demo-react-1-buildingcomponents/myshop/src/items/ItemListPage.jsx: Adjacent JSX elements must be wrapped in an enclosing tag. Did you want a JSX fragment <>...</>? (25:4)
```

The browser will also show the error:

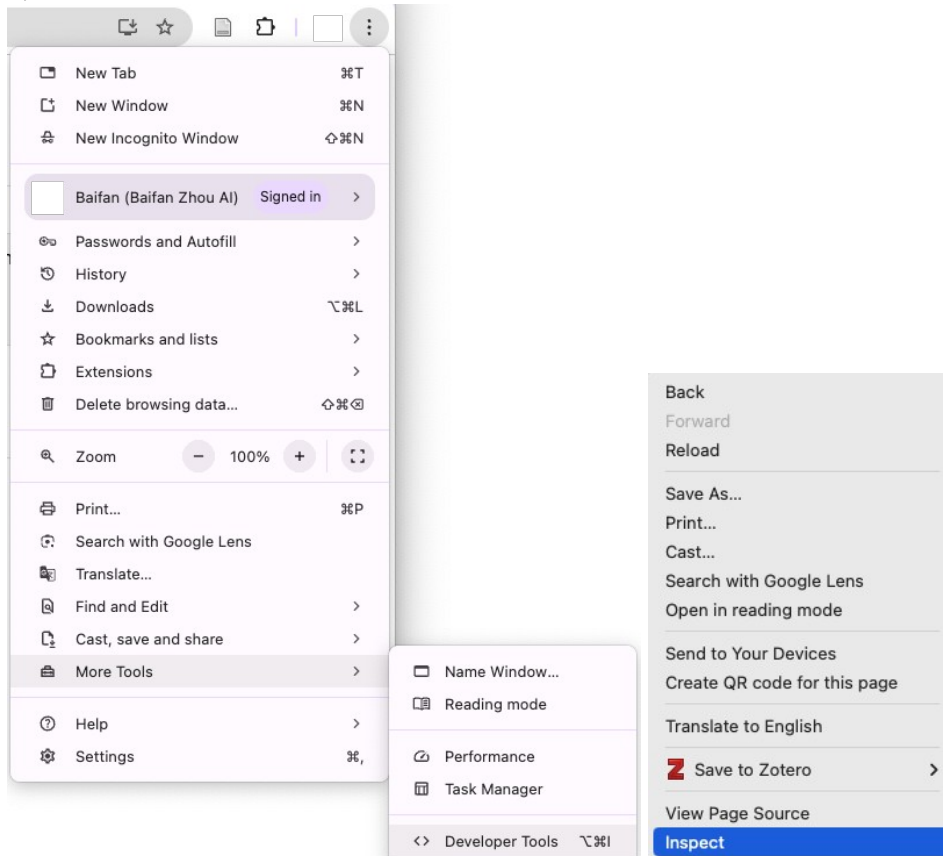


To fix it, simply correct it in the code. The browser will reload. There is no need to stop the frontend project and restart.

Use the Browser built-in Developer Tools

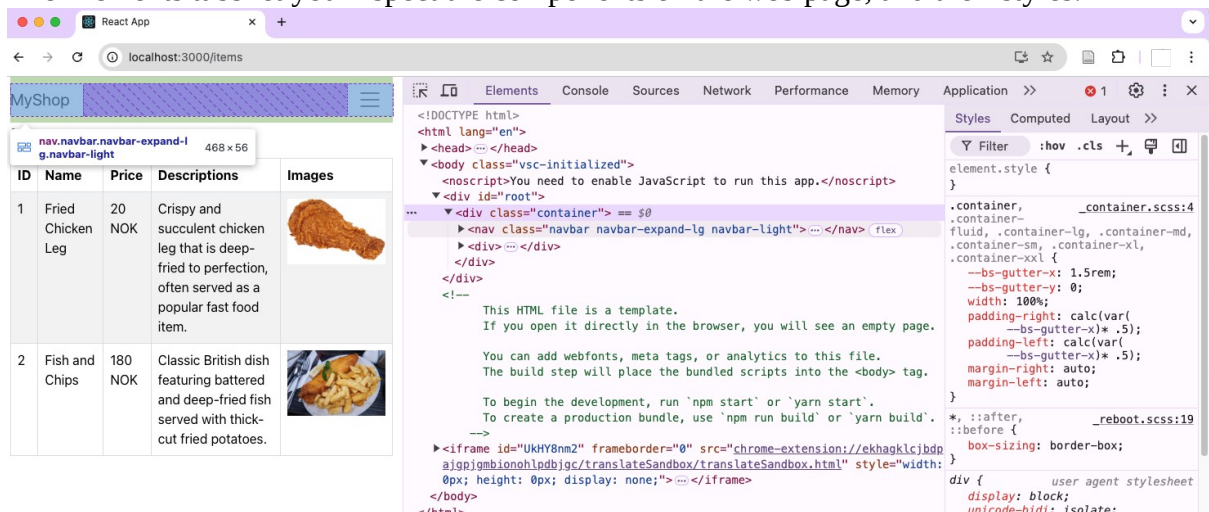
Multiple ways to open the browser built-in developer tools. We take Chrome as example. Other browsers are similar. Use Google to find out.

- 1, click the three dots → more tools → Developer Tools
- 2, right-click any webpage in an empty place → Inspect
- 3, Press F12.



Most commonly used are the Elements, Console, and Sources tabs.

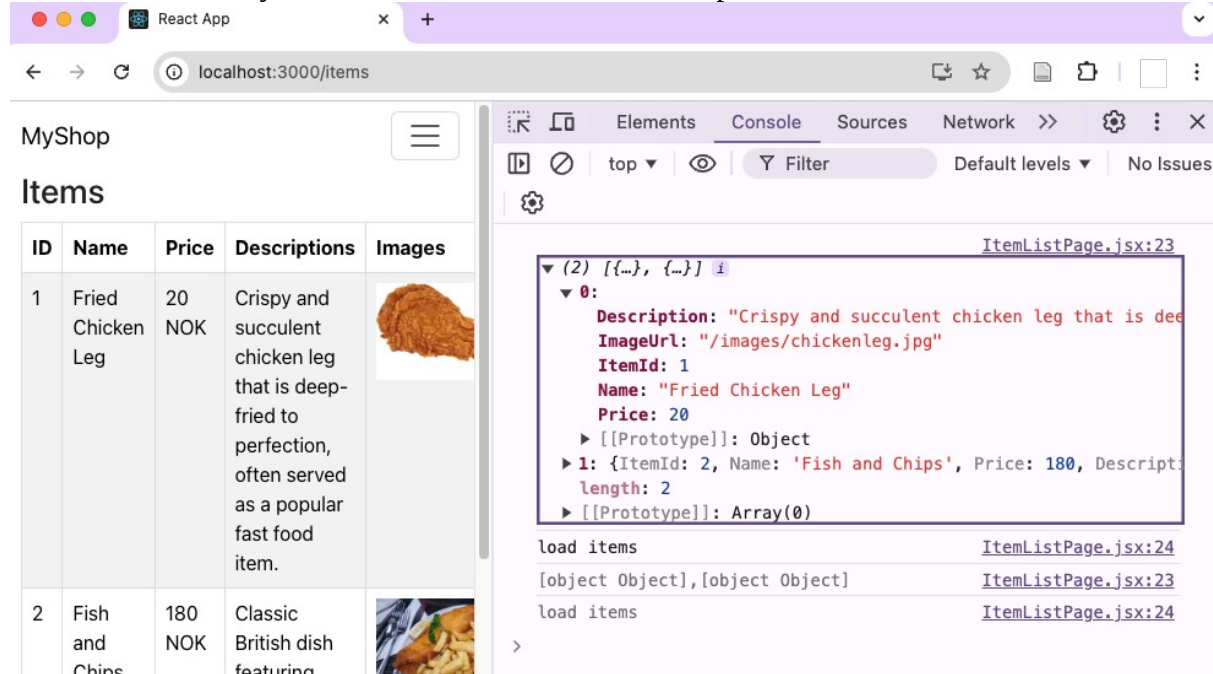
The Elements tabs let you inspect the components on the web page, and their styles.



The Console tab outputs the logs during the running.
For example, I add these lines in ItemListPage.jsx.

```
23 console.log(items)
24 console.log('load items')
```

In the Console tab, you will be able to see these are output in the console.

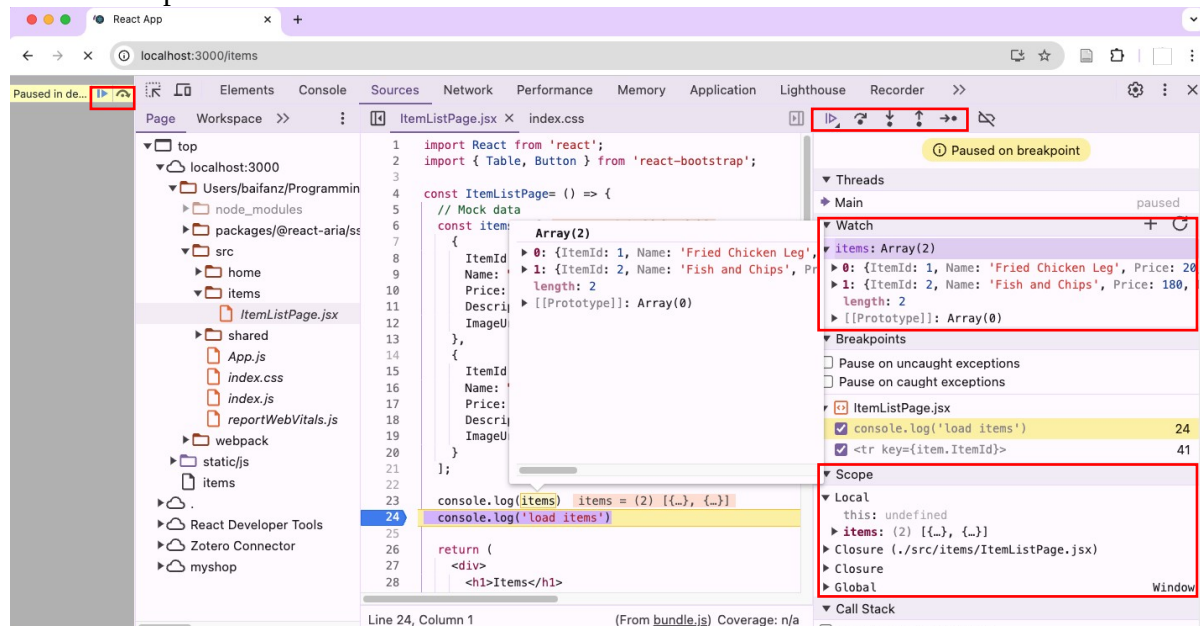


The Sources tab allows you to set breakpoints in the frontend code. In the following example, we add a breakpoint at line 24. If the ItemListPage is loaded, it stops at Line 24. Then we can hover over variables to see their content.

Use the buttons in red rectangles to control the debugging flow.

Use the Watch to add variables to observe.

Use the Scope to monitor all states/variables.

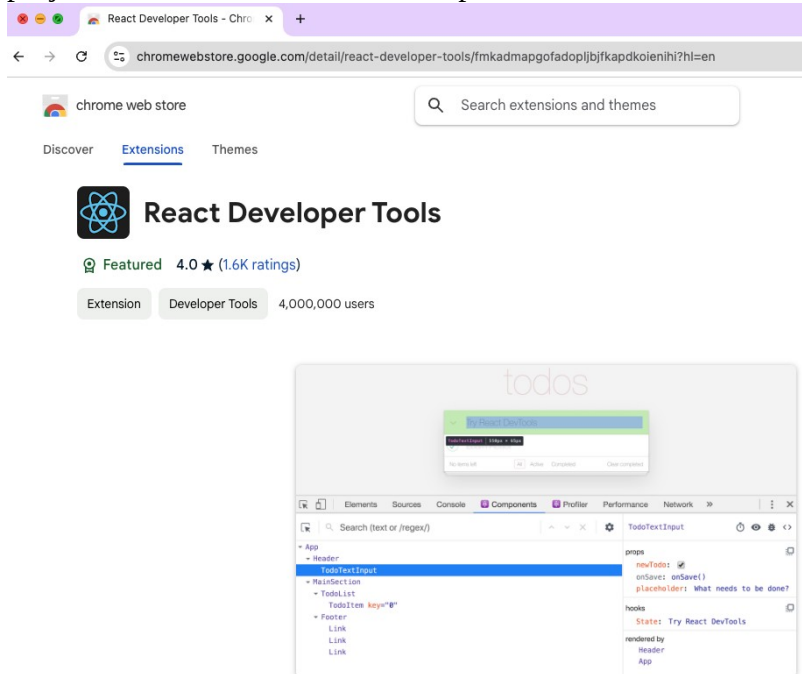


There are many good uses of the Developer Tools. You can explore a bit by yourself. 😊

Use the browser extension React Developer Tools

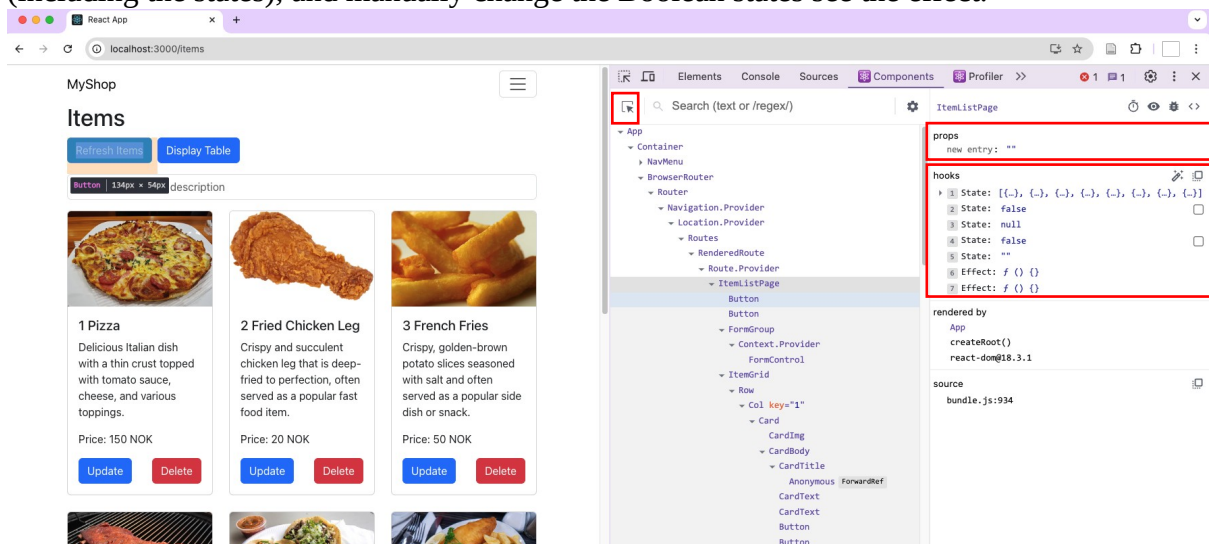
Install the browser extension: React Developer Tools. It should be available for many popular Browsers such as Chrome, Firefox, Edge, Safari (installation is different).

It's easier to see what the tool is capable if you have more complicated project. I use the project in Demo-React-7-AJAXHttpPutDelete to demonstrate the tool.

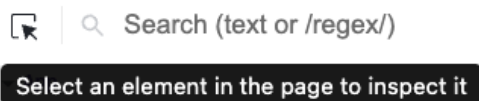


Two new tabs appear: Components, and Profiler.

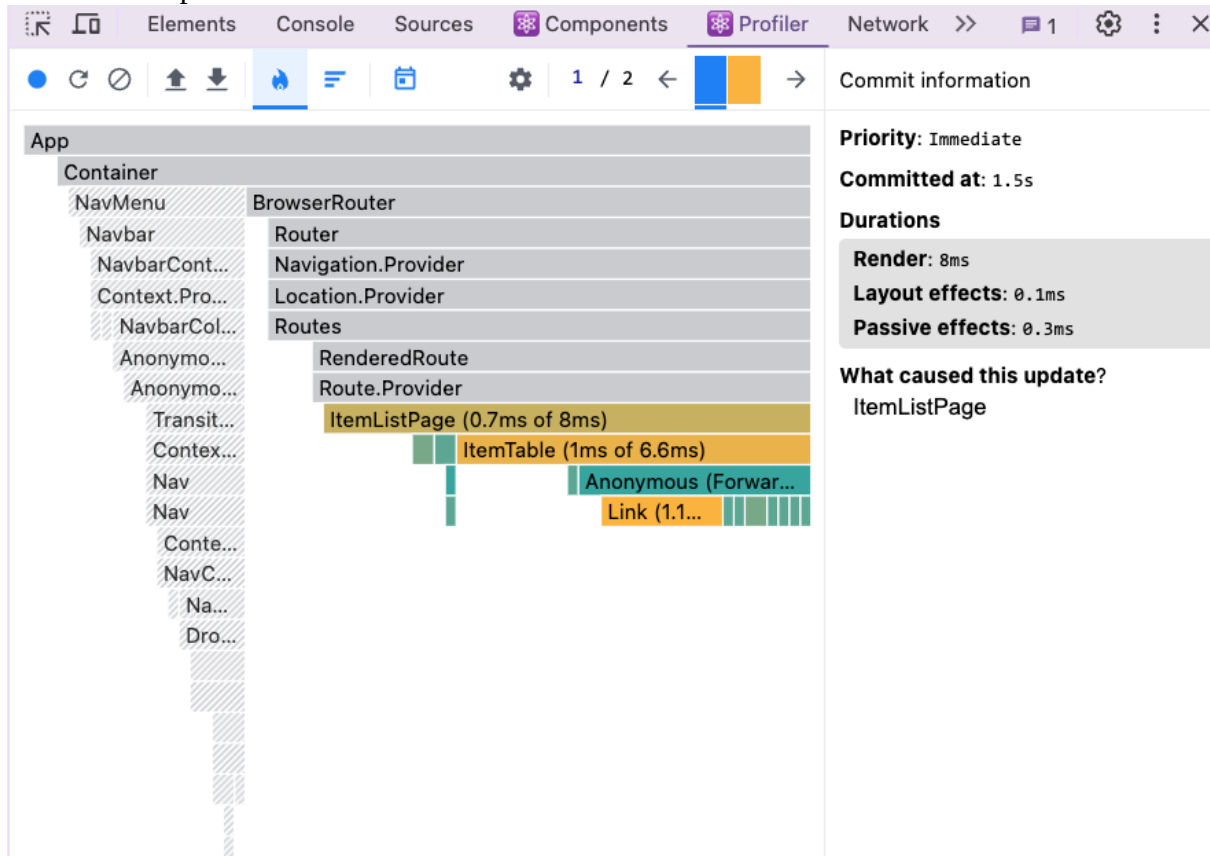
In the Components tab, you can see a tree of components. Click any of them, you can see the component highlighted on the web page, and view the input properties, all the hooks (including the states), and manually change the Boolean states to see the effect.



There is also an element picker in the red rectangle that allows you to pick up element, and a search bar for you to find elements.



The Profiler tab allows you to measure rendering performance of a React tree programmatically. You record a period of page running, and analyse the response time of different components.



Read more: <https://react.dev/learn/react-developer-tools>

Congratulations! You have learnt a handful of tools for debugging the React frontend project!

